

Testing Cheatsheet

Zwięzły cheatsheet obejmujący `pytest`, `unittest` i `unittest.mock`. Patrząc na tę stronę powinienś być w stanie napisać i zmockować większość typowych testów.

1. Pierwszy test w pytest

```
# test_math.py
def test_add():
    assert 1 + 1 == 2

def test_exception():
    with pytest.raises(ValueError, match="invalid"):
        int("abc") # rzuca ValueError
```

```
pytest                # uruchom wszystkie testy
pytest test_math.py   # jeden plik
pytest -k "add"        # testy pasujące do nazwy
pytest -x              # stop po pierwszym upadku
pytest -v              # verbose
```

2. Asercje w pytest

```
assert result == expected
assert result != bad_value
assert value > 0
assert item in collection
assert isinstance(obj, MyClass)

# porównanie float z tolerancją
assert result == pytest.approx(3.14, abs=0.01)

# sprawdzenie wyjątku
with pytest.raises(KeyError):
    d = {}
    d["missing"]
```

3. Fixtures

Fixture = funkcja dostarczająca dane / zasoby do testu.

```
import pytest

@pytest.fixture
def sample_user():
    return {"name": "Jan", "age": 30}
```

```
def test_user_name(sample_user):          # pytest wstrzyknie fixture
    assert sample_user["name"] == "Jan"
```

Scope	Kiedy tworzone	Przykład użycia
<code>function</code> (domyślny)	Przed każdym testem	lekkie obiekty
<code>class</code>	Raz na klasę testów	współdzielony stan
<code>module</code>	Raz na plik	połączenie DB
<code>session</code>	Raz na cały run	ciężki setup

```
@pytest.fixture(scope="module")
def db_connection():
    conn = create_connection()
    yield conn          # yield = teardown po testach
    conn.close()
```

4. Parametrize

Uruchom ten sam test z różnymi danymi:

```
@pytest.mark.parametrize("input,expected", [
    ("hello", 5),
    ("", 0),
    ("abc", 3),
])
def test_len(input, expected):
    assert len(input) == expected
```

5. Markery

```
@pytest.mark.slow
def test_heavy():
    ...

@pytest.mark.skip(reason="WIP")
def test_not_ready():
    ...

@pytest.mark.skipif(sys.platform == "win32", reason="Linux only")
def test_linux_only():
    ...
```

```
pytest -m slow          # uruchom tylko testy z markerem "slow"
pytest -m "not slow"    # pomiń testy slow
```

6. unittest — podstawy

```
import unittest

class TestMath(unittest.TestCase):
    def setUp(self):
        self.data = [1, 2, 3]

    def tearDown(self):
        self.data = None

    def test_sum(self):
        self.assertEqual(sum(self.data), 6)

    def test_contains(self):
        self.assertIn(2, self.data)

if __name__ == "__main__":
    unittest.main()
```

Metoda	Sprawdza
<code>assertEqual(a, b)</code>	<code>a == b</code>
<code>assertNotEqual(a, b)</code>	<code>a != b</code>
<code>assertTrue(x)</code>	<code>bool(x) is True</code>
<code>assertFalse(x)</code>	<code>bool(x) is False</code>
<code>assertIs(a, b)</code>	<code>a is b</code>
<code>assertIn(a, b)</code>	<code>a in b</code>
<code>assertIsInstance(a, cls)</code>	<code>isinstance(a, cls)</code>
<code>assertRaises(Exc)</code>	podnosi <code>Exc</code>

7. unittest.mock — Mock i MagicMock

```
from unittest.mock import Mock, MagicMock

# Mock – pusty obiekt, który rejestruje wywołania
mock = Mock()
mock.some_method(1, 2, key="val")
mock.some_method.assert_called_once_with(1, 2, key="val")

# MagicMock – Mock + magic methods (__len__, __getitem__, ...)
m = MagicMock()
m.__len__.return_value = 5
assert len(m) == 5
```

Atrybut / metoda	Opis
<code>mock.return_value</code>	wartość zwracana przy wywołaniu
<code>mock.side_effect</code>	funkcja/wyjątek/lista wyników
<code>mock.call_count</code>	ile razy wywołano
<code>mock.call_args</code>	ostatnie argumenty
<code>mock.assert_called()</code>	czy wywołano (min. raz)
<code>mock.assert_called_once()</code>	czy wywołano dokładnie raz
<code>mock.assert_not_called()</code>	czy nie wywołano
<code>mock.reset_mock()</code>	resetuje stan

```
# side_effect – różne wyniki dla kolejnych wywołań
mock = Mock(side_effect=[1, 2, ValueError("boom")])
mock() # → 1
mock() # → 2
mock() # → raises ValueError

# side_effect jako funkcja
mock = Mock(side_effect=lambda x: x * 2)
assert mock(3) == 6
```

8. patch — podmiana obiektów w testach

`patch` zamienia obiekt na Mock **na czas testu** i przywraca oryginał po zakończeniu.

Kluczowa zasada: patchujesz tam, gdzie obiekt jest **importowany**, nie tam, gdzie jest **zdefiniowany**.

```
from unittest.mock import patch

# Jako dekorator
@patch("myapp.services.requests.get")
def test_fetch(mock_get):
    mock_get.return_value.json.return_value = {"id": 1}
    result = fetch_user(1)
    assert result["id"] == 1
    mock_get.assert_called_once()

# Jako context manager
def test_fetch_v2():
    with patch("myapp.services.requests.get") as mock_get:
        mock_get.return_value.status_code = 200
        result = fetch_user(1)
        assert result is not None

# patch.object – patch konkretnego atrybutu obiektu
```

```
with patch.object(MyService, "call_api", return_value=42):
    assert MyService().call_api() == 42
```

9. Mockowanie typowych scenariuszy

```
# --- mockowanie metody klasy ---
class UserRepo:
    def get_by_id(self, user_id: int):
        ... # prawdziwe zapytanie do DB

@patch.object(UserRepo, "get_by_id", return_value={"name": "Jan"})
def test_get_user(mock_method):
    repo = UserRepo()
    assert repo.get_by_id(1)["name"] == "Jan"

# --- mockowanie zmiennej środowiskowej ---
@patch.dict("os.environ", {"API_KEY": "test-key-123"})
def test_env():
    import os
    assert os.environ["API_KEY"] == "test-key-123"

# --- mockowanie open() ---
from unittest.mock import mock_open

@patch("builtins.open", mock_open(read_data="file content"))
def test_read_file():
    with open("any_file.txt") as f:
        assert f.read() == "file content"

# --- mockowanie datetime.now ---
@patch("myapp.utils.datetime")
def test_time(mock_dt):
    mock_dt.now.return_value = datetime(2025, 1, 1, 12, 0)
    assert get_current_hour() == 12
```

10. conftest.py

Współdzielone fixture — pytest **automatycznie** importuje `conftest.py` z danego katalogu.

```
# conftest.py
import pytest

@pytest.fixture
def api_client():
    return TestClient(app)

@pytest.fixture(autouse=True)    # automatycznie dla KAŻDEGO testu
def reset_db():
    db.clear()
    yield
    db.clear()
```

11. Testowanie wyjątków (pytest vs unittest)

```
# pytest
def test_division_by_zero():
    with pytest.raises(ZeroDivisionError):
        1 / 0

# unittest
class TestDiv(unittest.TestCase):
    def test_division_by_zero(self):
        with self.assertRaises(ZeroDivisionError):
            1 / 0
```

12. Struktura projektu testowego

```
myproject/
├── src/
│   └── myapp/
│       ├── __init__.py
│       ├── services.py
│       └── utils.py
├── tests/
│   ├── conftest.py           # wspólne fixtures
│   ├── test_services.py
│   └── test_utils.py
├── pyproject.toml
└── pytest.ini               # lub sekcja [tool.pytest] w pyproject.toml
```

```
# pytest.ini
[pytest]
testpaths = tests
addopts = -v --tb=short
markers =
    slow: marks tests as slow
```

Powiązane pliki

- [PyTest: Przykłady i Scenariusze Testowania](#)
- [PyTest: Mockowanie](#)
- [Fixture overload — wyjaśnienie](#)
- [Patch i pytest.mark.asyncio w testach](#)
- [Pełny przykład testu z mockowaniem MagicMock](#)